# RBFL with Trainable Gating and Feedback

A Measured Win on Cumulative Parity and Its Limits

Reproducible study of the BFL repository

October 10, 2026

**Abstract**

We present a recurrent network of eight Boolean MUX units with discrete connections, registered state, and a trainable enable route for each unit. Training retains the lexicographic *argmin* criterion of binary-global BFL: it minimizes total errors first, then new errors on previously correct predictions, and finally the number of changed configuration bits. Across five seeds of a cumulative parity task, the `rbfl_used_m_trainable_feedback` variant reached 100% bit and exact sequence accuracy on 400 sequences of unseen lengths and another 100% on 400 sequences with a shifted distribution. The Elman RNN control reached 81.08% ± 19.31% bit accuracy on the unseen-length test. The ablation with trainable gating but no temporal persistence reached 54.92% ± 1.61%. The success is specific to this construction: on the historical static task, the evaluated RBFL variant reached 73.44%, below the 90.00% of binary-global BFL and the 99.29% of the MLP. We provide code, deterministic splits, per-seed results, and the limits of this comparison.

## 1 Scope and provenance

The reference result for binary-global BFL is 90.3066% mean test accuracy across 50 seeds, archived in commit `53c2f46`. We recompiled that commit's benchmark unchanged: across 20 seeds, it produced 90.6132% ± 1.6145%. The original model uses 32 routes selected by five input bits, 16 Boolean features, and a majority vote of active features. Training enumerates proposals of up to three bit flips, masks them with the current example's `USED` features, and evaluates the entire training group for that route. It compares (ERR, FORGET, CHANGE_COST) lexicographically. In that code, `CHANGE` denotes a configuration proposal; it is not the error mask.

This study evaluates a different architecture with explicit temporal memory. Accordingly, *winning variant* means only the best RBFL measured on the temporal parity task under this protocol; it does not imply superiority on static tasks or real data. The row identifiers are:

- `rbfl_used_m_trainable_feedback`: parity winner;
- `rbfl_used_m_fixed1_feedback`: fixed-gate control;
- `rbfl_used_m_trainable_no_feedback`: memory ablation.

## 2 RBFL architecture

The state has eight bits, $H_t = (Q_{1,t}, \ldots, Q_{8,t})$. Each $A_{i,t}$ and $B_{i,t}$ input selects one of twenty sources: constants 0 and 1, the ten bits of $X_t$, or the eight bits of $H_t$. Every unit reads the same $H_t$ before $H_{t+1}$ is written, so there are no combinational cycles. For each MUX,

$$M_{i,t} = (A_{i,t} \wedge \neg Q_{i,t}) \vee (B_{i,t} \wedge Q_{i,t}), \tag{1}$$

$$Q_{i,t+1} = Q_{i,t} \oplus \big[\mathrm{USED}_{M,i,t} \wedge (Q_{i,t} \oplus M_{i,t})\big]. \tag{2}$$

In the winning variant, $\mathrm{USED}_{M,i,t}$ also selects a Boolean source from those twenty choices. Its value does not depend on the target during inference. The output is a selected bit of $H_{t+1}$, optionally inverted. The initial state is zero and resets between sequences, while persisting across steps within a sequence.

**Table 1. Feedback MUX before gating.**

| $A$ | $B$ | $M$ | $A$ | $B$ | $M$ |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 1 | $Q$ |
| 1 | 0 | $\neg Q$ | 1 | 1 | 1 |

The usage mask records $U^A_{i,t} = \mathrm{USED}_{M,i,t} \wedge \neg Q_{i,t}$ and $U^B_{i,t} = \mathrm{USED}_{M,i,t} \wedge Q_{i,t}$. These observations are aggregated into $\mathrm{USED}_G$ across training trajectories. The search also evaluates rewires of inactive routes; otherwise, a never-used route could not be discovered. Each source index occupies five bits, and a rewire's cost is the Hamming distance between the old and new indices. The winning variant has 124 trainable configuration bits and occupies 44 bytes as a measured C++ object.

# 3 Lexicographic greedy search

We define $\mathrm{CHANGE}_{\mathrm{ERROR}} = P_\theta(X) \oplus Y$, separately from the proposed parameter change $\Delta$. Each candidate resets $H_0$, replays the *entire* collection of training sequences, and obtains $E_\Delta = Y \oplus P_{\theta_\Delta}(X)$. With $E_0 = Y \oplus P_\theta(X)$, the implemented criterion is

$$\Delta^* = \arg\min_{\Delta}^{\mathrm{lex}} \left( \sum E_\Delta,\ \sum (\neg E_0 \wedge E_\Delta),\ \mathrm{popcount}(\theta \oplus \theta_\Delta) \right). \tag{3}$$

The current model is compared with the tuple $(\sum E_0, 0, 0)$; a proposal is retained only if it improves that tuple, and is otherwise reverted. RBFL uses no gradients, SGD, Adam, or floating-point weights. The search tries one complete rewire or one output or initialization edit per candidate, for up to four greedy iterations. This neighborhood differs from the historical BFL's flips of up to three feature bits. `FORGET` only breaks ties after total errors are equal.

# 4 Experimental protocol

The historical static benchmark contains all 1,024 distinct ten-bit vectors, shuffled with `std::mt19937(seed)` and split into 600 training and 424 test examples. A second split of the same domain reserves 600/200/224 for training, validation, and test. We retain binary-global BFL from the historical commit and a 16-hidden-unit MLP with 193 `float64` parameters, 500 Adam epochs, and a learning rate of 0.03. The Elman RNN control has eight hidden units and 161 `float64` parameters; it uses full BPTT, Adam, 500 epochs, and a learning rate of 0.01.

In the temporal task, the relevant bit $x_{t,0}$ is a fair Bernoulli variable and the other nine bits are distractors. The target is $y_t = \bigoplus_{s=0}^{t} x_{s,0}$. Each seed generates 160 training and 40 validation sequences of lengths 4–8. The final test contains 80 sequences of lengths 9–12 and a separate shifted test of 80 sequences of those lengths with $\Pr(x_{t,0} = 1) = 0.2$. There are five seeds. The final temporal test uses a separate random stream fixed after preliminary exploration and before these figures were obtained. RBFL parameters are not optimized on test data.

Inference metrics exclude reading and tokenization: they use 10,000 warmup calls and five batches of 100,000 calls, each for a complete sequence. Nanoseconds per step divide the time by mean sequence length. Persistent memory is the model object; temporary memory is a lower bound for inference data, excluding allocator metadata. Measurements use `g++ 13.3` with `-O3 -march=native` on an AMD Ryzen 7 5700U. CPU instruction counters were unavailable.

# 5 Results

**Table 2. Historical static task, five-seed mean; RBFL with $\mathrm{USED}_M = 1$.**

| Model | Train | Test ± SD | Training (ms) | Inference (ns/step) |
|---|---|---|---|---|
| Binary-global BFL | 96.13% | 90.00% ± 1.82 | 242.19 | 14.13 |
| RBFL, $\mathrm{USED}_M = 1$ | 76.27% | 73.44% ± 1.19 | 18.88 | 45.78 |
| MLP, 16 hidden units | 100.00% | 99.29% ± 0.47 | 160.65 | 425.32 |
| Elman RNN, 8 hidden units | 99.83% | 97.31% ± 1.40 | 110.40 | 347.15 |

The 600/200/224 static split confirms the negative result: test accuracy is 90.63% for BFL, 73.39% for RBFL with $\mathrm{USED}_M = 1$, 99.38% for the MLP, and 97.23% for the RNN. The favorable temporal result therefore cannot be extrapolated to a general improvement over the original BFL.

**Table 3. Cumulative parity, five-seed mean on unseen lengths.**

| Model | Test bit ± SD | Exact | Shifted | Train (ms) |
|---|---|---|---|---|
| MLP, no memory | 50.56% ± 2.23 | 0.25% | 52.04% | 224.89 |
| Elman RNN | 81.08% ± 19.31 | 54.75% | 80.79% | 148.85 |
| RBFL, $\mathrm{USED}_M = 1$ | 58.82% ± 0.96 | 8.75% | 64.94% | 38.08 |
| **RBFL, learned gate + memory** | **100.00% ± 0.00** | **100.00%** | **100.00%** | 57.37 |
| RBFL, learned gate, no memory | 54.92% ± 1.61 | 0.25% | 59.88% | 34.87 |

**Table 4. Temporal inference cost; temporary bytes are a lower bound.**

| Model | Model (B) | Temporary (B) | Total (B) | ns/step |
|---|---|---|---|---|
| MLP, no memory | 1544 | 128 | 1672 | 393.05 |
| Elman RNN | 1288 | 940 | 2228 | 276.51 |
| RBFL, $\text{USED}_M = 1$ | 44 | 116 | 160 | 29.70 |
| **RBFL, trainable $\text{USED}_M$ + feedback** | 44 | 116 | 160 | 31.88 |
| RBFL, trainable $\text{USED}_M$, no feedback | 44 | 116 | 160 | 31.22 |

The winning variant improves on the RNN mean by 18.92 percentage points on this parity test, although the RNN varies substantially across seeds. The difference from the same RBFL without persistence is 45.08 points: feedback provides a measurable advantage under this protocol. Removing `FORGET` from the $\text{USED}_M = 1$ control did not change accuracy across these five seeds; neither did allowing trainable $Q_0$ in that neighborhood. Per-seed details are in `results/ablation.csv`.

# 6 Discussion and limitations

The task favors a compact Boolean solution: input-dependent gating can implement a conditional toggle of Q. This suggests a plausible mechanism, but does not establish an advantage for the architecture on other memory problems. We measured five seeds and one synthetic temporal rule; we did not test noisy sequences, real data, or an exhaustive hyperparameter search. The architectures do not share parameter or training-time budgets, so the cost comparison characterizes these particular implementations. The RNN has gradients and greater nominal capacity, while RBFL faces a discrete search limited to one change per iteration. Memory figures omit allocator overhead; timing depends on the machine and code. Historical BFL has no comparable temporal state and therefore is not a parity control with memory.

# 7 Per-seed results and checks

**Table 5. Bit accuracy on unseen lengths by seed.**

| Seed | Winning RBFL | Elman RNN | RBFL, no memory |
|---|---|---|---|
| 1 | 100.00% | 99.05% | 57.35% |
| 2 | 100.00% | 54.96% | 54.37% |
| 3 | 100.00% | 93.76% | 52.46% |
| 4 | 100.00% | 97.35% | 54.70% |
| 5 | 100.00% | 60.26% | 55.70% |

These values explain the RNN's high dispersion: three seeds exceeded 93%, while two stayed near chance. This table does not replace a study with more seeds and tasks. Implementation tests checked all four truth-table rows for both Q values, simultaneous updates across units, retention when $\text{USED}_M = 0$, temporal persistence, recording of the selected A and B routes, and the identity $\text{CHANGE}_{\text{ERROR}} = P \oplus Y$. A training test verified that a useful proposal changes parameters and that no positive-cost edit is accepted when error is already zero. All 120 rows of the final trace satisfy $\text{ERR}_{\text{best}} \leq \text{ERR}_{\text{before}}$. The RBFL inference executable receives inputs only; labels appear only in supervised evaluation.

# 8 Reproduction

The code and data are in the `research/rbfl_vs_ann/` research directory. Run:

```
bash scripts/reproduce.sh 5 20
```

This command recompiles the four models, checks RBFL, and generates the result, summary, ablation, and trace CSV files. Regenerate the PDF with `bash scripts/build_paper.sh`. Variant labels were clarified after measurement; nonexistent splits appear as `NA` in the CSV files, with all measurements retained. The winning variant's identifier is:

`rbfl_used_m_trainable_feedback`.